NullPointerExample

1. Aim

To demonstrate NullPointerExample exception handling in Java using appropriate techniques.

2. Description

Demonstrates handling a NullPointerException when a method is called on a null reference.

3. Algorithm

1. Start the program.
2. Set the String reference to null.
3. Print the name.
4. Call length() on the null reference, causing an exception.
5. Catch NullPointerException and display suitable messages.
6. End the program.

4. UML Diagram

NullPointerExample
+ main(args: String[]): void
+ handles exception and prints result

5. Source Code with Comments

```
public class NullPointerExample {
    public static void main(String[] args) {
        try {
            String name = null;
            System.out.println("Name: " + name);
            // Calling a method on a null reference
            System.out.println("Length: " + name.length());
        } catch (NullPointerException e) {
            System.out.println("Caught NullPointerException!");
            System.out.println("Cannot perform an operation on a null object.");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac NullPointerExample.java
java NullPointerExample
```

7. Output Screenshot

```
Name: null<br/>Caught NullPointerException!<br/>Cannot perform an operation on a null object.
```

8. Result

The NullPointerExample program demonstrates the expected exception-handling behavior.

ArrayIndexExample

1. Aim

To demonstrate ArrayIndexExample exception handling in Java using appropriate techniques.

2. Description

Demonstrates handling ArrayIndexOutOfBoundsException by accessing an invalid array index.

3. Algorithm

1. Start the program.
2. Create an array with three values.
3. Access an index outside the valid range.
4. Catch the exception and display a clear message.
5. End the program.

4. UML Diagram

ArrayIndexExample
+ main(args: String[]): void
+ handles exception and prints result

5. Source Code with Comments

```
public class ArrayIndexExample {
    public static void main(String[] args) {
        int[] values = {10, 20, 30};
        try {
            System.out.println("Value at index 1: " + values[1]);
            System.out.println("Trying index 5...");
            System.out.println(values[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Caught ArrayIndexOutOfBoundsException!");
            System.out.println("Index 5 is outside the array range 0-2.");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac ArrayIndexExample.java
java ArrayIndexExample
```

7. Output Screenshot

```
Value at index 1: 20<br/>Trying index 5...<br/>Caught ArrayIndexOutOfBoundsException!<br/>Index 5
is outside the array range 0-2.
```

8. Result

The ArrayIndexExample program demonstrates the expected exception-handling behavior.

NumberFormat

1. Aim

To demonstrate NumberFormat exception handling in Java using appropriate techniques.

2. Description

Demonstrates NumberFormatException when a non-numeric string is converted to an integer.

3. Algorithm

1. Start the program.
2. Store a non-numeric string.
3. Attempt to convert it to an integer.
4. Catch NumberFormatException and print a message.
5. End the program.

4. UML Diagram

NumberFormat
+ main(args: String[]): void
+ handles exception and prints result

5. Source Code with Comments

```
public class NumberFormat {
    public static void main(String[] args) {
        String invalidNumber = "12A";
        try {
            int number = Integer.parseInt(invalidNumber);
            System.out.println("Number: " + number);
        } catch (NumberFormatException e) {
            System.out.println("Conversion failed for: " + invalidNumber);
            System.out.println("Please provide digits only.");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac NumberFormat.java
java NumberFormat
```

7. Output Screenshot

```
Conversion failed for: 12A<br/>Please provide digits only.
```

8. Result

The NumberFormat program demonstrates the expected exception-handling behavior.

StringIndex

1. Aim

To demonstrate StringIndex exception handling in Java using appropriate techniques.

2. Description

Demonstrates StringIndexOutOfBoundsException when an invalid string index is accessed.

3. Algorithm

1. Start the program.
2. Create the string "Hello".
3. Attempt to access character at index 7.
4. Catch the exception and display a message.
5. End the program.

4. UML Diagram

StringIndex
+ main(args: String[]): void
+ handles exception and prints result

5. Source Code with Comments

```
public class StringIndex {
    public static void main(String[] args) {
        String text = "Hello";
        try {
            System.out.println("Text: " + text);
            System.out.println("Character at index 7: " + text.charAt(7));
        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("Caught StringIndexOutOfBoundsException!");
            System.out.println("Valid indexes are 0 through 4.");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac StringIndex.java
java StringIndex
```

7. Output Screenshot

```
Text: Hello<br/>Caught StringIndexOutOfBoundsException!<br/>Valid indexes are 0 through 4.
```

8. Result

The StringIndex program demonstrates the expected exception-handling behavior.

IllegalArgument

1. Aim

To demonstrate IllegalArgument exception handling in Java using appropriate techniques.

2. Description

Demonstrates validation with IllegalArgumentException when a negative age is supplied.

3. Algorithm

1. Start the program.
2. Call setAge() with -5.
3. Check whether age is negative.
4. Throw and catch IllegalArgumentException.
5. End the program.

4. UML Diagram

IllegalArgument
+ main(args: String[]): void
+ handles exception and prints result

5. Source Code with Comments

```
public class IllegalArgument {
    public static void setAge(int age) {
        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative.");
        }
        System.out.println("Age accepted: " + age);
    }
    public static void main(String[] args) {
        try {
            setAge(-5);
        } catch (IllegalArgumentException e) {
            System.out.println("Invalid age: " + e.getMessage());
        }
    }
}
```

6. Compilation and Execution Commands

```
javac IllegalArgument.java
java IllegalArgument
```

7. Output Screenshot

```
Invalid age: Age cannot be negative.
```

8. Result

The IllegalArgument program demonstrates the expected exception-handling behavior.

ClassCastExample

1. Aim

To demonstrate ClassCastExample exception handling in Java using appropriate techniques.

2. Description

Demonstrates ClassCastException caused by an incompatible type cast.

3. Algorithm

1. Start the program.
2. Store a String in an Object reference.
3. Attempt to cast the object to Integer.
4. Catch ClassCastException and show a message.
5. End the program.

4. UML Diagram

ClassCastExample
+ main(args: String[]): void
+ handles exception and prints result

5. Source Code with Comments

```
public class ClassCastExample {
    public static void main(String[] args) {
        Object value = "Java exception demo";
        try {
            Integer number = (Integer) value;
            System.out.println(number);
        } catch (ClassCastException e) {
            System.out.println("Caught ClassCastException!");
            System.out.println("A String object cannot be cast to Integer.");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac ClassCastExample.java
java ClassCastExample
```

7. Output Screenshot

```
Caught ClassCastException!<br/>A String object cannot be cast to Integer.
```

8. Result

The ClassCastExample program demonstrates the expected exception-handling behavior.

FileNotFound

1. Aim

To demonstrate FileNotFound exception handling in Java using appropriate techniques.

2. Description

Demonstrates FileNotFoundException when a requested file cannot be located.

3. Algorithm

1. Start the program.
2. Attempt to open a file that does not exist.
3. Catch FileNotFoundException.
4. Display a helpful message and end.

4. UML Diagram

FileNotFound
+ main(args: String[]): void
+ handles exception and prints result

5. Source Code with Comments

```
import java.io.FileReader;
import java.io.FileNotFoundException;

public class FileNotFound {
    public static void main(String[] args) {
        try (FileReader reader = new FileReader("missing_demo_file.txt")) {
            System.out.println("File opened successfully.");
        } catch (FileNotFoundException e) {
            System.out.println("Unable to open missing_demo_file.txt");
            System.out.println("Check the file name and location.");
        } catch (java.io.IOException e) {
            System.out.println("Error while closing the file.");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac FileNotFound.java
java FileNotFound
```

7. Output Screenshot

```
Unable to open missing_demo_file.txt<br/>Check the file name and location.
```

8. Result

The FileNotFound program demonstrates the expected exception-handling behavior.

IOException

1. Aim

To demonstrate IOException exception handling in Java using appropriate techniques.

2. Description

Demonstrates IOException handling during file input/output.

3. Algorithm

1. Start the program.
2. Open a reader for a file.
3. Close the reader and attempt another read.
4. Catch IOException and display a message.
5. End the program.

4. UML Diagram

IOException
+ main(args: String[]): void
+ handles exception and prints result

5. Source Code with Comments

```
import java.io.FileReader;
import java.io.IOException;

public class IOException {
    public static void main(String[] args) {
        try {
            FileReader reader = new FileReader("test.txt");
            reader.close();
            reader.read(); // Reading after close causes an I/O error
        } catch (IOException e) {
            System.out.println("Caught IOException during file access.");
            System.out.println("The read operation could not be completed.");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac IOException.java
java IOException
```

7. Output Screenshot

```
Caught IOException during file access.<br/>The read operation could not be completed.
```

8. Result

The IOException program demonstrates the expected exception-handling behavior.

CustomException

1. Aim

To demonstrate CustomException exception handling in Java using appropriate techniques.

2. Description

Demonstrates a custom checked exception for voting eligibility.

3. Algorithm

1. Start the program.
2. Define InvalidAgeException extending Exception.
3. Call checkVotingEligibility(16).
4. Throw and catch the custom exception for age below 18.
5. Display the message and end.

4. UML Diagram

CustomException
+ main(args: String[]): void
+ handles exception and prints result

5. Source Code with Comments

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class CustomException {
    public static void checkVotingEligibility(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Must be 18 or older to vote.");
        }
        System.out.println("Eligible to vote.");
    }
    public static void main(String[] args) {
        try {
            checkVotingEligibility(16);
        } catch (InvalidAgeException e) {
            System.out.println("Voting check failed: " + e.getMessage());
        }
    }
}
```

6. Compilation and Execution Commands

```
javac CustomException.java
java CustomException
```

7. Output Screenshot

```
Voting check failed: Must be 18 or older to vote.
```

8. Result

The CustomException program demonstrates the expected exception-handling behavior.